

QA AUTOMATION • INTERVIEW PREP

Playwright

Interview Notes

JavaScript / TypeScript • Questions, answers & code examples

147

Questions

24

Topics

90+

Code examples

Prepared by

Piyush Kolhe

Contents

How to use these notes: read the question, try to answer it out loud first, then check the answer and run the example. Repeat until each answer comes out naturally.

1	Playwright Fundamentals	11 Q	3
2	Browser, Context and Page	6 Q	4
3	Locators	10 Q	5
4	Actions	8 Q	6
5	Assertions and Waiting	8 Q	8
6	Hooks and Fixtures	5 Q	9
7	Frames and Dialogs	5 Q	10
8	Tabs, Windows and Popups	4 Q	11
9	Page Object Model	5 Q	12
10	API Testing	6 Q	13
11	Authentication and Storage State	4 Q	14
12	Network Interception and Mocking	5 Q	14
13	Screenshots, Video and Trace	5 Q	15
14	Reports, Tags and Annotations	7 Q	16
15	Parallelism, Retries and Projects	6 Q	17
16	Configuration	5 Q	18
17	Tables, Lists and Dynamic Elements	5 Q	19
18	Mouse and Keyboard	5 Q	20
19	CI/CD and Git	5 Q	20
20	Debugging and Common Errors	5 Q	22
21	Scenario-Based Interview Questions	9 Q	22
22	Advanced Interview Questions	11 Q	24
23	Practical Coding Questions	5 Q	26
24	Interview Quick Revision	2 Q	27

1

Playwright Fundamentals

11 questions

Q1

What is Playwright?

Playwright is an open-source web automation and end-to-end testing framework developed by Microsoft. It supports Chromium, Firefox, and WebKit and provides auto-waiting, assertions, API testing, network interception, tracing, screenshots, video, and parallel execution.

Q2

What is Playwright Test?

Playwright Test is Playwright's own test runner. It provides test execution, fixtures, hooks, assertions, projects, retries, parallel execution, reporters, screenshots, videos, and tracing.

Q3

Which browsers does Playwright support?

Chromium, Firefox, and WebKit. It can also automate branded Chromium browsers such as Chrome and Edge using browser channels.

Q4

Which languages does Playwright support?

Official Playwright APIs are available for JavaScript/TypeScript, Python, Java, and .NET.

Q5

What are the major advantages of Playwright?

Built-in auto-waiting, cross-browser support, isolated browser contexts, reliable locators, multiple-tab handling, iframe support, API testing, network mocking, screenshots, video, tracing, and parallel execution.

Q6

What is the difference between Selenium and Playwright?

Selenium traditionally uses WebDriver and browser drivers, while Playwright communicates with supported browsers using its own automation mechanisms. Playwright provides built-in auto-waiting, BrowserContext isolation, modern locator APIs, easy popup handling, tracing, network interception, and built-in test-runner features.

Q7

How do you install Playwright?

Create a project with `npm init playwright@latest`, or install `@playwright/test` using `npm`. Then install the required browsers.

EXAMPLE

```
npm init playwright@latest  
  
# or manually  
npm install -D @playwright/test  
npx playwright install
```

Q8 How do you execute all Playwright tests?

Use the test command.

EXAMPLE

```
npx playwright test
```

Q9 How do you execute a specific test file?

Pass the file path to the test command.

EXAMPLE

```
npx playwright test tests/login.spec.js
```

Q10 How do you run tests in headed mode?

Add the `--headed` flag so the browser window is visible.

EXAMPLE

```
npx playwright test --headed
```

Q11 How do you debug a test?

Run with `--debug` (opens Playwright Inspector) or place `await page.pause()` inside the test.

EXAMPLE

```
npx playwright test --debug

// or inside the test
await page.pause();
```

2 Browser, Context and Page

6 questions

Q12 What is a Page?

A Page represents a single browser tab or page.

Q13 What is Browser?

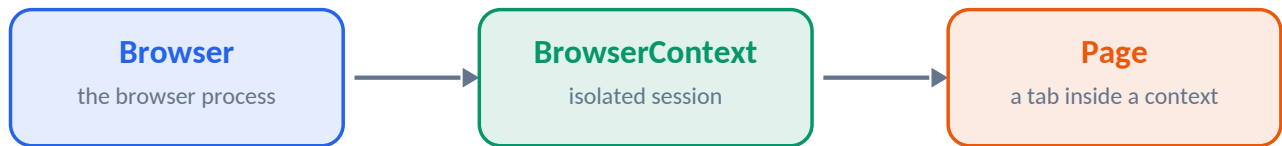
Browser represents the browser instance launched by Playwright.

Q14 What is BrowserContext?

BrowserContext is an isolated browser session. It has separate cookies, local storage, session storage, and authentication state.

Q15 Difference between Browser, BrowserContext and Page?

Browser is the browser process; BrowserContext is an isolated session inside the browser; Page is a tab inside a context.



Q16 How do you create a browser context manually?

Call `newContext()` on the browser and then `newPage()` on the context.

EXAMPLE

```
const context = await browser.newContext();  
const page = await context.newPage();
```

Q17 Why are BrowserContexts useful?

They provide isolation between tests or users without launching a completely separate browser process for every session.

3

Locators

10 questions

Q18 What is a Locator?

A Locator identifies an element and provides Playwright's auto-waiting and retry behavior for actions and assertions.

Q19 Common Playwright locator methods?

`getByRole`, `getByText`, `getByLabel`, `getByPlaceholder`, `getByAltText`, `getByTitle`, `getByTestId`, and `locator`.

Q20 What is `getByRole()`?

It locates an element using its accessible role and optionally its accessible name.

EXAMPLE

```
page.getByRole('button', { name: 'Login' })
```

Q21 What is `getByText()`?

It locates elements based on visible text.

EXAMPLE

```
page.getByText('Login')
```

Q22 What is getByLabel()?

It locates form controls associated with a label.

EXAMPLE

```
page.getByLabel('Username')
```

Q23 What is getByPlaceholder()?

It locates an input using its placeholder text.

EXAMPLE

```
page.getByPlaceholder('Enter username')
```

Q24 What is getByTestId()?

It locates an element using a test ID attribute such as data-testid.

EXAMPLE

```
// <button data-testid="login-button">  
page.getByTestId('login-button')
```

Q25 What is locator()?

It creates a locator using CSS, XPath, or another supported selector expression.

EXAMPLE

```
page.locator('#username') // CSS  
page.locator('//input[@id="username"]') // XPath
```

Q26 Which locator strategy should be preferred?

Prefer stable, user-facing locators such as role, label, text, and test ID when appropriate. Use CSS or XPath when those are not suitable.

Q27 CSS vs XPath?

Both are supported. CSS is often concise, while XPath is useful for certain relationships and text-based expressions. The best selector is the one that is stable and meaningful for the application.

4

Actions

8 questions

Q28 How do you enter text?

Use fill().

EXAMPLE

```
await page.locator('#username').fill('piyush');
```

Q29 Difference between fill() and pressSequentially()?

fill() sets the complete input value after clearing it. pressSequentially() sends the characters one by one and is useful when the application depends on keyboard events.

EXAMPLE

```
await page.locator('#search').fill('playwright');  
await page.locator('#search').pressSequentially('playwright');
```

Q30 How do you click an element?

Use click().

EXAMPLE

```
await locator.click();
```

Q31 How do you select a native dropdown?

Use selectOption().

EXAMPLE

```
await page.locator('#country').selectOption({ label: 'India' });
```

Q32 How do you check a checkbox?

Use check().

EXAMPLE

```
await locator.check();
```

Q33 How do you uncheck a checkbox?

Use uncheck().

EXAMPLE

```
await locator.uncheck();
```

Q34 How do you upload a file?

Use setInputFiles() on the file input.

EXAMPLE

```
await locator.setInputFiles('C:/files/test.pdf');
```

Q35 How do you download a file?

Wait for the download event, click the download control, then save the returned Download object using `saveAs()`.

EXAMPLE

```
const downloadPromise = page.waitForEvent('download');
await page.getByText('Download').click();
const download = await downloadPromise;
await download.saveAs('files/' + download.suggestedFilename());
```

5 Assertions and Waiting

8 questions

Q36 What is an assertion?

An assertion verifies that the actual application state matches the expected state.

Q37 Common Playwright assertions?

`toBeVisible`, `toBeHidden`, `toBeEnabled`, `toBeDisabled`, `toHaveText`, `toContainText`, `toHaveValue`, `toBeChecked`, `toHaveURL`, and `toHaveTitle`.

EXAMPLE

```
await expect(locator).toBeVisible();
await expect(locator).toHaveText('Welcome');
await expect(page).toHaveTitle('Dashboard');
```

Q38 How do you verify a URL?

Use `toHaveURL` with a regex or an exact URL.

EXAMPLE

```
await expect(page).toHaveURL(/dashboard/);
```

Q39 Difference between `toHaveText()` and `toContainText()`?

`toHaveText()` verifies the expected text; `toContainText()` verifies that the expected text is contained in the actual text.

EXAMPLE

```
// actual text: "Welcome, Piyush"
await expect(locator).toContainText('Piyush');           // passes
await expect(locator).toHaveText('Welcome, Piyush');     // full match
```

Q40 What is auto-waiting?

Playwright automatically waits for elements to become ready for actions and retries web-first assertions until the condition is satisfied or the timeout is reached.

Q41 Is `waitForTimeout()` recommended?

Generally no. Fixed waits slow tests and can still be unreliable. Prefer locator actions, assertions, or condition-based waits.

Q42 How do you wait for a URL?

Use `page.waitForURL()` when explicit URL synchronization is needed.

EXAMPLE

```
await page.waitForURL('**/dashboard');
```

Q43 How do you wait for an element?

Use locator actions or assertions where possible. `locator.waitFor()` can be used when explicit waiting for an element state is genuinely required.

EXAMPLE

```
await page.locator('#loader').waitFor({ state: 'hidden' });
```

6

Hooks and Fixtures

5 questions

Q44 What are Playwright hooks?

Hooks are lifecycle methods used for common setup and cleanup: `beforeAll`, `beforeEach`, `afterEach`, and `afterAll`.

EXAMPLE

```
test.beforeEach(async ({ page }) => {  
  await page.goto('/login');  
});  
  
test.afterEach(async ({ page }) => {  
  // cleanup  
});
```

Q45 Difference between `beforeAll` and `beforeEach`?

`beforeAll` runs once for the relevant scope. `beforeEach` runs before every test.

Q46 Why is `page` normally not used in `beforeAll`?

The built-in page fixture is test-scoped. For normal page navigation and setup, use `beforeEach`. If worker-level setup is required, use an appropriate worker-scoped fixture or create the required context explicitly.

Q47 What is a fixture?

A fixture provides reusable test setup and objects such as page, browser, context, and request.

EXAMPLE

```
test('example', async ({ page, context, request }) => { /* ... */ });
```

Q48 What is the request fixture?

It provides an APIRequestContext for API testing.

EXAMPLE

```
test('api', async ({ request }) => {  
  const res = await request.get('/users');  
});
```

7

Frames and Dialogs

5 questions

Q49 How do you handle an iframe?

Use frameLocator().

EXAMPLE

```
const frame = page.frameLocator('#myFrame');  
await frame.locator('#username').fill('piyush');
```

Q50 Selenium vs Playwright iframe handling?

Selenium commonly uses switchTo().frame(). Playwright can interact through frameLocator() without switching the driver context in the same way.

Q51 How do you handle an alert?

Listen for the dialog event and call accept() or dismiss().

EXAMPLE

```
page.on('dialog', async dialog => await dialog.accept());  
// or: await dialog.dismiss();
```

Q52 How do you handle a prompt?

Use dialog.accept('value') to provide prompt text.

EXAMPLE

```
page.on('dialog', async dialog => await dialog.accept('Piyush'));
```

Q53 How do you get alert text?

Use `dialog.message()`.

EXAMPLE

```
page.on('dialog', async dialog => {  
  console.log(dialog.message());  
  await dialog.accept();  
});
```

8 Tabs, Windows and Popups

4 questions

Q54 How do you handle a new tab?

Create a popup promise before clicking, then await it.

EXAMPLE

```
const popupPromise = page.waitForEvent('popup');  
await page.getByText('Open').click();  
const popup = await popupPromise;
```

Q55 Does Playwright have getWindowHandles()?

No. Playwright works with Page objects rather than Selenium-style window handles.

Q56 How do you handle multiple pages?

Use the `BrowserContext`'s `pages` collection and inspect each Page as needed.

EXAMPLE

```
const pages = context.pages();
```

Q57 How do you select a popup based on title?

Wait for the popup, wait for it to load, read its title, and compare it with the expected title. For multiple pages, iterate through `context.pages()` and inspect titles.

EXAMPLE

```
for (const p of context.pages()) {  
  if ((await p.title()) === 'Expected Title') {  
    // work with p  
  }  
}
```

Q58 What is Page Object Model?

POM is a design pattern where page-specific locators and actions are kept in separate page classes/files, while test files focus on scenarios.

Q59 Advantages of POM?

Reusability, maintainability, reduced duplication, cleaner tests, and separation of test logic from page interaction logic.

Q60 What should a Page Object contain?

Usually the page reference, stable locators, and reusable business/page actions. Assertions can be kept in tests or placed in page objects when they represent page-specific behavior.

Q61 Example POM?

Create a LoginPage class with username, password, and login-button locators plus a login() method. Import the class into the test and call the method.

EXAMPLE

```
// LoginPage.js
export class LoginPage {
  constructor(page) {
    this.page = page;
    this.username = page.locator('#user-name');
    this.password = page.locator('#password');
    this.loginBtn = page.locator('#login-button');
  }
  async login(user, pass) {
    await this.username.fill(user);
    await this.password.fill(pass);
    await this.loginBtn.click();
  }
}

// login.spec.js
import { LoginPage } from './LoginPage';
test('login', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await page.goto('/');
  await loginPage.login('standard_user', 'secret_sauce');
});
```

Q62 export default vs module.exports?

In an ES module project, export default is imported using import. In CommonJS, use module.exports and require(). Do not mix both module systems unnecessarily.

EXAMPLE

```
// ES Module
export default LoginPage;
import LoginPage from './LoginPage';

// CommonJS
module.exports = LoginPage;
const LoginPage = require('./LoginPage');
```

10 API Testing

6 questions

Q63 Can Playwright perform API testing?

Yes. Playwright provides APIRequestContext and the request fixture.

Q64 How do you send a GET request?

Use request.get().

EXAMPLE

```
const response = await request.get(url);
```

Q65 How do you send a POST request?

Use request.post() with a data payload.

EXAMPLE

```
const response = await request.post(url, {
  data: { name: 'Piyush', job: 'QA' }
});
```

Q66 How do you verify status code?

Use response.status() with an expect.

EXAMPLE

```
expect(response.status()).toBe(200);
```

Q67 How do you read JSON response?

Use response.json().

EXAMPLE

```
const body = await response.json();
```

Q68 Can UI and API tests be combined?

Yes. API calls can be used for setup, cleanup, authentication, or direct API validation alongside UI tests.

11 Authentication and Storage State

4 questions

Q69 How do you avoid logging in before every test?

Authenticate once and save storage state, then reuse it for tests.

Q70 What is storageState?

It stores authentication-related browser state such as cookies and local storage.

Q71 Example of saving storage state?

Save the state after login, then point the configuration to that file.

EXAMPLE

```
// save after login
await context.storageState({ path: 'auth.json' });

// playwright.config.js
use: { storageState: 'auth.json' }
```

Q72 Why use authentication state?

It reduces repeated login steps, speeds up tests, and keeps test cases focused on their actual scenarios.

12 Network Interception and Mocking

5 questions

Q73 Can Playwright intercept network requests?

Yes. Use `page.route()` to intercept matching requests.

EXAMPLE

```
await page.route('**/api/users', route => route.continue());
```

Q74 What is `route.continue()`?

It allows the intercepted request to continue normally, optionally after modifying it.

EXAMPLE

```
await route.continue({ headers: { ...route.request().headers(), 'x-test': '1' } });
```

Q75 What is route.abort()?

It blocks the request.

EXAMPLE

```
await page.route('**/*.png', route => route.abort());
```

Q76 What is route.fulfill()?

It provides a mocked response instead of sending the request to the real backend.

EXAMPLE

```
await page.route('**/api/users', route =>
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([{ id: 1, name: 'Piyush' }])
  })
);
```

Q77 Why mock APIs?

To test UI behavior independently of unstable, unavailable, slow, or expensive backend services.

13 Screenshots, Video and Trace

5 questions

Q78 How do you take a screenshot?

Use page.screenshot(). Use fullPage: true for a full-page screenshot.

EXAMPLE

```
await page.screenshot({ path: 'screenshot.png' });
await page.screenshot({ path: 'full.png', fullPage: true });
```

Q79 How do you record video?

Configure video: 'on' or 'retain-on-failure' in Playwright configuration.

EXAMPLE

```
use: { video: 'retain-on-failure' }
```

Q80 What is Trace Viewer?

Trace Viewer is a debugging tool that lets you inspect test actions, snapshots, screenshots, network information, and timing.

Q81 How do you enable tracing?

A common configuration is trace: 'on-first-retry'.

EXAMPLE

```
use: { trace: 'on-first-retry' }
```

Q82 How do you open a trace?

Use the show-trace command.

EXAMPLE

```
npx playwright show-trace trace.zip
```

14 Reports, Tags and Annotations

7 questions

Q83 How do you generate an HTML report?

Use the HTML reporter and then open it after the test execution.

EXAMPLE

```
// playwright.config.js
reporter: 'html'

// terminal
npx playwright show-report
```

Q84 What reporters are available?

Common reporters include list, line, dot, HTML, JSON, and JUnit.

EXAMPLE

```
reporter: [['html'], ['junit', { outputFile: 'results.xml' }]]
```

Q85 How do you tag a test?

Add a tag such as @smoke in the test title and filter with --grep.

EXAMPLE

```
test('login @smoke', async ({ page }) => { /* ... */ });

npx playwright test --grep @smoke
```

Q86 How do you skip a test?

Use test.skip() or test.describe.skip().

EXAMPLE

```
test.skip('not ready', async ({ page }) => { /* ... */ });
```

Q87 What is test.fail()?

It marks a test as expected to fail. If it unexpectedly passes, Playwright reports that condition.

Q88 What is test.fixme()?

It marks a test as needing a fix and skips its execution.

Q89 What is test.slow()?

It marks a test as slow and triples its timeout.

EXAMPLE

```
test('big flow', async ({ page }) => {  
  test.slow();  
});
```

15 Parallelism, Retries and Projects

6 questions

Q90 Does Playwright support parallel execution?

Yes. Playwright Test uses workers and can run tests in parallel.

Q91 How do you configure workers?

Use the workers option in Playwright configuration.

EXAMPLE

```
workers: 4
```

Q92 How do you run tests serially?

Use serial mode when test order/dependency requires it.

EXAMPLE

```
test.describe.configure({ mode: 'serial' });
```

Q93 How do you configure retries?

Use the retries option in the configuration or project settings.

EXAMPLE

```
retries: 2
```

Q94 What are projects?

Projects let you run the same test suite under different configurations such as Chromium, Firefox, WebKit, mobile profiles, or different environments.

Q95 Why use projects?

They provide clean cross-browser, device, environment, or configuration matrices without duplicating tests.

EXAMPLE

```
projects: [  
  { name: 'chromium', use: { ...devices['Desktop Chrome'] } },  
  { name: 'firefox', use: { ...devices['Desktop Firefox'] } },  
  { name: 'webkit', use: { ...devices['Desktop Safari'] } },  
]
```

16 Configuration

5 questions

Q96 What is playwright.config.js?

It is the central configuration file for test directory, timeout, retries, workers, projects, reporter, browser settings, base URL, screenshots, video, trace, and other test options.

Q97 How do you set a base URL?

Configure baseURL and then use relative paths.

EXAMPLE

```
use: { baseURL: 'https://example.com' }  
  
await page.goto('/login');
```

Q98 How do you configure screenshots?

Set the screenshot option.

EXAMPLE

```
use: { screenshot: 'only-on-failure' }
```

Q99 How do you configure trace?

Set the trace option.

EXAMPLE

```
use: { trace: 'on-first-retry' }
```

Q100 How do you configure browser projects?

Define projects in the configuration and specify browser/device settings for each project.

EXAMPLE

```
projects: [{ name: 'chromium', use: { browserName: 'chromium' } }]
```

17

Tables, Lists and Dynamic Elements

5 questions

Q101 How do you get all text from matching elements?

Use `allTextContents()` to get an array of text values.

EXAMPLE

```
const items = await page.locator('.item').allTextContents();
```

Q102 How do you count elements?

Use `count()`.

EXAMPLE

```
const total = await locator.count();
```

Q103 How do you access a specific matching element?

Use `nth(index)`, `first()`, or `last()` where appropriate.

EXAMPLE

```
locator.nth(2); locator.first(); locator.last();
```

Q104 How do you handle dynamic tables?

Locate the table, identify rows using stable text or attributes, then locate cells relative to the matching row. Avoid brittle positional selectors when stable identifiers are available.

EXAMPLE

```
const row = page.locator('tr').filter({ hasText: 'Piyush' });  
await row.locator('td').nth(2).click();
```

Q105 How do you find a row containing text?

Use a locator filtered by text when multiple rows exist.

EXAMPLE

```
page.locator('tr').filter({ hasText: 'Piyush' })
```

18

Mouse and Keyboard

5 questions

Q106 How do you perform hover?

Use `hover()`.

EXAMPLE

```
await locator.hover();
```

Q107 How do you double-click?

Use `dblclick()`.

EXAMPLE

```
await locator.dblclick();
```

Q108 How do you right-click?

Use `click()` with the `right` button.

EXAMPLE

```
await locator.click({ button: 'right' });
```

Q109 How do you press a keyboard key?

Use `keyboard.press()`.

EXAMPLE

```
await page.keyboard.press('Enter');
```

Q110 How do you drag and drop?

Use `dragTo()` when supported, or use mouse actions for custom drag behavior.

EXAMPLE

```
await source.dragTo(target);
```

19

CI/CD and Git

5 questions

Q111 Can Playwright run in Jenkins?

Yes. Install Node.js and project dependencies on the Jenkins agent, run `npm ci`, install required browsers/dependencies, execute the tests, and publish the generated report/artifacts.

EXAMPLE

```
npm ci
npx playwright install --with-deps
npx playwright test
```

Q112 How do you run Playwright in CI?

Use a reproducible dependency installation such as `npm ci`, install browsers or use an appropriate Playwright container/image, then execute the test command.

Q113 What artifacts are useful in CI?

HTML reports, screenshots, videos, traces, test results, logs, and optionally JUnit/JSON reports.

Q114 How do you run only smoke tests in CI?

Tag smoke tests and run with `--grep`.

EXAMPLE

```
npx playwright test --grep @smoke
```

Q115 How do you run Playwright from GitHub Actions?

Use a workflow that checks out code, installs Node dependencies, installs Playwright browsers/dependencies, runs tests, and uploads the report/artifacts.

EXAMPLE

```
name: Playwright Tests
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
```

20

Debugging and Common Errors

5 questions

Q116 What does 'No tests found' mean?

Playwright did not discover tests matching the command/configuration. Check the test file path, filename pattern, testDir, and whether the file actually contains a Playwright test() call.

Q117 Why does strict mode violation occur?

A locator matched more than one element while the operation expected one element. Make the locator more specific or intentionally select the required element.

EXAMPLE

```
// fails if 3 buttons match
await page.locator('button').click();

// fix
await page.getByRole('button', { name: 'Save' }).click();
```

Q118 Why does a locator timeout occur?

The element may not exist, may be hidden, may be blocked, the locator may be incorrect, the page may not have reached the expected state, or the application may be slower than the configured timeout.

Q119 How do you debug a locator?

Use Playwright Inspector, page.pause(), trace viewer, browser developer tools, and locator assertions such as toBeVisible() or count().

Q120 Why can a title assertion fail after opening a popup?

The test may have captured the wrong popup/page. Wait for the popup event, then inspect the popup's URL/title, or inspect context.pages() when multiple pages are involved.

21

Scenario-Based Interview Questions

9 questions

Q121 Login must happen once for 50 tests. What would you do?

Use a setup/authentication project to create and save storage state, then reuse the state in the dependent tests.

EXAMPLE

```
projects: [
  { name: 'setup', testMatch: /auth\.setup\.js/ },
  { name: 'chromium',
    use: { storageState: 'auth.json' },
    dependencies: ['setup'] },
]
```

Q122 An element appears after an API response. How would you handle it?

Prefer a web assertion such as `toBeVisible()`. If necessary, wait for a specific response using `page.waitForResponse()` and then validate the UI.

EXAMPLE

```
await expect(locator).toBeVisible();

// or
const resp = page.waitForResponse('**/api/data');
await page.getByText('Load').click();
await resp;
```

Q123 A test is flaky in CI but passes locally. What would you check?

Check timing assumptions, hard waits, environment differences, network dependency, shared test data, parallelism, browser versions, resource constraints, and trace/video artifacts.

Q124 The same button text appears multiple times. What would you do?

Use a more specific locator based on role/name, container, label, test ID, or another stable attribute. Avoid blindly using `nth()` unless position is truly part of the requirement.

EXAMPLE

```
page.locator('#cart').getByRole('button', { name: 'Remove' })
```

Q125 An iframe is nested inside another iframe. How would you handle it?

Chain frame locators.

EXAMPLE

```
page.frameLocator('#outer').frameLocator('#inner').locator('#field')
```

Q126 You need to test a UI without calling the real backend. What would you do?

Intercept the API with `page.route()` and provide a controlled response with `route.fulfill()`.

EXAMPLE

```
await page.route('**/api/orders', r => r.fulfill({ json: [] }));
```

Q127 You need to test a file download and verify its name. What would you do?

Wait for the download event, click the control, obtain the Download object, and validate properties such as `suggestedFilename()` before saving if needed.

EXAMPLE

```
const downloadPromise = page.waitForEvent('download');
await page.getByText('Export').click();
const download = await downloadPromise;
expect(download.suggestedFilename()).toBe('report.csv');
```

Q128 A test opens a new tab. What is the correct order?

Start waiting for the popup before the click, perform the click, await the popup promise, wait for it to load if necessary, then interact with it.

EXAMPLE

```
const popupPromise = page.waitForEvent('popup'); // 1. wait first
await page.getByText('Open').click();           // 2. click
const popup = await popupPromise;               // 3. get popup
await popup.waitForLoadState();                  // 4. load
```

Q129 How would you design a Playwright framework for a large project?

Use a clear folder structure, POM or component objects, fixtures, test data management, environment configuration, API helpers, utilities, authentication setup, projects, tagging, reporting, trace/screenshots, and CI integration.

EXAMPLE

```
project/
├─ tests/           (spec files)
├─ pages/           (page objects)
├─ fixtures/        (custom fixtures)
├─ test-data/
├─ utils/           (helpers, API helpers)
├─ playwright.config.js
└─ .github/workflows/
```

22 Advanced Interview Questions

11 questions

Q130 What is web-first assertion?

It is an assertion that waits and retries until the expected web state is reached or the timeout expires, making tests less timing-sensitive.

EXAMPLE

```
await expect(locator).toBeVisible(); // web-first (retries)
```

Q131 What is the difference between locator and element handle?

Locators are the preferred high-level abstraction and re-resolve elements when used. ElementHandle represents a particular DOM element and is generally less resilient for normal test interactions.

Q132 Why are locators preferred over ElementHandle?

Locators support auto-waiting, retryability, and resilient interaction patterns. Element handles can become stale when the DOM changes.

Q133 What is a worker in Playwright Test?

A worker is a separate process used by the test runner to execute tests. Tests can be distributed across workers for parallel execution.

Q134 What is test isolation?

Each test should run independently with isolated state and data so that one test does not affect another.

Q135 How do BrowserContexts help test isolation?

Each context has independent cookies, storage, permissions, and session state, allowing tests to simulate separate users or sessions.

Q136 What is a custom fixture?

A custom fixture extends Playwright's built-in fixture system to provide reusable setup, objects, or teardown logic to tests.

EXAMPLE

```
import { test as base } from '@playwright/test';
import { LoginPage } from './LoginPage';

export const test = base.extend({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
});
```

Q137 What is a global setup?

Global setup is project-level initialization that can run before the test suite, such as preparing shared resources or authentication state. It should be used carefully because excessive global state can reduce test isolation.

EXAMPLE

```
// playwright.config.js
globalSetup: require.resolve('./global-setup')
```

Q138 What is the difference between test data and configuration?

Configuration controls how tests run, such as base URL and browsers. Test data represents inputs and expected business values used by test cases.

Q139 How can you make Playwright tests maintainable?

Use stable locators, POM/component objects, reusable fixtures, API helpers, isolated test data, clear naming, minimal hard waits, meaningful assertions, and CI diagnostics.

Q140 How do you reduce Playwright execution time?

Use parallel workers appropriately, reuse authentication state, avoid unnecessary waits, use API setup instead of repetitive UI setup, reduce redundant browser launches, and split projects/tests intelligently.

23

Practical Coding Questions

5 questions

Q141 Write a simple login test.

EXAMPLE

```
import { test, expect } from '@playwright/test';

test('login', async ({ page }) => {
  await page.goto('https://www.saucedemo.com/');
  await page.locator('#user-name').fill('standard_user');
  await page.locator('#password').fill('secret_sauce');
  await page.locator('#login-button').click();
  await expect(page).toHaveURL(/inventory/);
});
```

Q142 Write a popup-handling example.

EXAMPLE

```
const popupPromise = page.waitForEvent('popup');
await page.getByText('Open').click();
const popup = await popupPromise;
await popup.waitForLoadState();
console.log(await popup.title());
```

Q143 Write an iframe example.

EXAMPLE

```
const frame = page.frameLocator('#myFrame');
await frame.locator('#username').fill('piyush');
```

Q144 Write an API GET test.

EXAMPLE

```
test('GET API', async ({ request }) => {
  const response = await request.get('https://api.example.com/users');
  expect(response.status()).toBe(200);
  const body = await response.json();
  console.log(body);
});
```

Q145 Write a file upload example.

EXAMPLE

```
await page.locator('#fileInput').setInputFiles('C:/test/file.pdf');
```

24

Interview Quick Revision

2 questions

Q146

Most important Playwright concepts to remember

Browser → Context → Page; Locator → Action → Assertion; auto-waiting; web-first assertions; fixtures; hooks; POM; frameLocator; popup event; request fixture; storageState; route interception; trace viewer; projects; retries; workers; reports; CI/CD.

Q147

One-minute Playwright interview answer

Playwright is a Microsoft-developed end-to-end automation framework supporting Chromium, Firefox, and WebKit. It provides reliable locator APIs, automatic waiting, web-first assertions, isolated browser contexts, multi-page and iframe handling, API testing, network interception, authentication state, screenshots, video, tracing, parallel execution, and CI-friendly reporting. I use JavaScript/TypeScript with Playwright Test, POM, fixtures, reusable utilities, and CI integration.